

UNIVERSIDAD NACIONAL DE SAN AGUSTIN
Facultad de Ingenieria de Produccion y Servicios
Escuela Profesional de Ingenieria de Sistemas

Informe de Laboratorio

Clasificacion y Reconocimiento de Objetos con UI

Aplicacion multimedia desarrollada con Python, Streamlit, OpenCV y YOLOv8

Asignatura	Computacion Grafica, Vision Computacional y Multimedia
Tema	Clasificacion y Reconocimiento
Tipo de practica	Individual / Grupal maximo 3
Integrante(s)	Jeremy Perez / completar integrantes del grupo
Fecha	2025B

Resumen

El proyecto implementa una interfaz web para cargar imagenes JPG, PNG o WEBP, reconocer objetos con YOLOv8, visualizar cajas delimitadoras, mostrar metricas y exportar evidencias en imagen y CSV. El sistema cubre el flujo de adquisicion, procesamiento, clasificacion, reconocimiento e interpretacion de resultados.

Indice

1. Introduccion
2. Objetivos
3. Descripcion general del programa
4. Tecnologias utilizadas
5. Funcionamiento del sistema
6. Funciones creadas
7. Modificaciones realizadas
8. Ventajas y desventajas
9. Evidencias y video de demostracion
10. Cuestionario
11. Conclusiones
12. Referencias

1. Introduccion

El presente laboratorio desarrolla una aplicacion multimedia enfocada en la clasificacion y reconocimiento de objetos dentro de imagenes digitales. Para ello se implemento una interfaz web que permite al usuario subir imagenes en formato JPG, JPEG, PNG o WEBP y ejecutar un modelo de deteccion de objetos basado en YOLOv8.

La aplicacion muestra el resultado visual mediante cajas delimitadoras y genera una tabla con la clase del objeto, el nivel de confianza y las coordenadas de ubicacion. De esta manera se evidencia el procesamiento de imagenes desde la adquisicion hasta la interpretacion final.

2. Objetivos

- Crear una aplicacion multimedia orientada al reconocimiento y clasificacion de objetos.
- Aplicar tecnicas de vision computacional sobre imagenes digitales.
- Implementar una interfaz grafica clara para cargar imagenes y visualizar resultados.
- Generar evidencias mediante imagen anotada, tabla de resultados y archivo CSV.
- Explicar las tecnologias y algoritmos empleados en el proyecto.

3. Descripcion general del programa

El programa desarrollado se denomina Detector de Objetos con YOLOv8. Su finalidad es reconocer objetos comunes presentes en una imagen, clasificarlos segun una clase preentrenada y localizar su posicion mediante cajas delimitadoras.

El usuario interactua con una UI desarrollada en Streamlit. Desde esta interfaz puede subir una imagen JPG, PNG o WEBP, configurar el umbral de confianza, modificar el umbral IoU/NMS, seleccionar el tamano de inferencia y ejecutar la deteccion. Una vez procesada la imagen, la aplicacion muestra la imagen original, la imagen con detecciones, metricas generales, tabla detallada, grafico de conteo por clase y botones para descargar evidencias.

4. Tecnologias utilizadas

Tecnologia	Uso dentro del proyecto
Python	Lenguaje principal de desarrollo.
Streamlit	Creacion de la interfaz web interactiva.
YOLOv8 / Ultralytics	Algoritmo de deteccion, clasificacion y localizacion de objetos.
OpenCV	Dibujo de cajas, etiquetas y procesamiento visual.
Pillow	Lectura y conversion de imagenes JPG, PNG y WEBP.
Pandas	Creacion de tabla de detecciones y exportacion CSV.
NumPy	Manejo de matrices de imagen.

5. Funcionamiento del sistema

El flujo de procesamiento implementado es el siguiente:

- 1 Adquisicion de imagen: el usuario sube una imagen desde la UI mediante `st.file_uploader`.
- 2 Normalizacion: la imagen se convierte a RGB. Si tiene transparencia, se coloca sobre un fondo blanco.

- 3 Inferencia: el modelo YOLOv8 analiza la imagen completa y predice objetos, clases, confianza y coordenadas.
- 4 Filtrado: se eliminan detecciones con confianza inferior al umbral configurado.
- 5 Postprocesamiento: se aplica IoU/NMS para reducir cajas duplicadas sobre el mismo objeto.
- 6 Visualizacion: OpenCV dibuja las cajas delimitadoras, etiquetas y niveles de confianza.
- 7 Reporte de resultados: Streamlit presenta metricas, tabla, grafico y opciones de descarga.

Comando de ejecucion de la UI:

```
streamlit run app.py
```

6. Funciones creadas

Funcion	Descripcion
load_yolo_model()	Carga el modelo YOLOv8 usando cache para evitar recargas innecesarias.
pil_to_rgb_array()	Convierte la imagen subida a matriz RGB compatible con YOLO y OpenCV.
run_detection()	Ejecuta la deteccion y devuelve imagen anotada, tabla y conteo.
draw_detection()	Dibuja la caja delimitadora, clase y confianza sobre la imagen.
image_to_bytes()	Convierte la imagen procesada a bytes para su descarga.
dataframe_to_csv_bytes()	Convierte la tabla de detecciones en CSV descargable.
save_evidence()	Guarda la imagen detectada y CSV dentro de la carpeta runs/.

7. Modificaciones realizadas

A diferencia de un ejemplo basico por consola, este proyecto fue adaptado a una interfaz web completa. Las principales modificaciones fueron:

- Se creo una UI visual con Streamlit.
- Se agrego carga de archivos JPG, JPEG, PNG y WEBP.
- Se agrego conversion de imagenes con transparencia a RGB.
- Se implementaron controles para confianza, IoU/NMS y tamano de inferencia.
- Se agregaron metricas automaticas de objetos detectados, clases distintas y confianza promedio.
- Se implemento una tabla con clase, confianza, coordenadas y area.
- Se agrego descarga de imagen procesada y CSV.
- Se agrego una seccion dentro de la UI para explicar el funcionamiento del laboratorio.
- Se mantuvo una version opcional por consola para imagen, video y webcam.

8. Ventajas y desventajas

Aspecto	Ventajas	Desventajas
YOLOv8	Rapido, adecuado para aplicaciones interactivas, detecta y clasifica en una sola etapa.	Puede fallar con objetos pequenos, tapados, borrosos o fuera de las clases entrenadas.
Streamlit	Permite crear una UI clara sin desarrollar frontend complejo.	Depende de un servidor local y no es ideal para apps moviles nativas.

Aspecto	Ventajas	Desventajas
OpenCV	Facilita el procesamiento y dibujo sobre imagenes.	El manejo de colores puede requerir conversiones entre BGR y RGB.
Modelo preentrenado	Permite probar sin entrenar desde cero.	Limita el reconocimiento a clases conocidas por el dataset base.

9. Evidencias y video de demostracion

Para la entrega se debe grabar un video demostrando la ejecucion y funcionamiento de la aplicacion. Se recomienda mostrar los siguientes puntos:

- 1 Apertura del proyecto en VSCode.
- 2 Ejecucion de streamlit run app.py.
- 3 Apertura de la interfaz web.
- 4 Carga de una imagen JPG, PNG o WEBP.
- 5 Configuracion de confianza e IoU.
- 6 Resultado con cajas delimitadoras.
- 7 Tabla de detecciones y grafico.
- 8 Descarga del CSV o imagen procesada.
- 9 Carpeta runs/ con evidencias guardadas.

Enlace a scripts creados: colocar aqui el enlace de Drive, GitHub o repositorio donde se suba la carpeta del proyecto.

Checklist de cumplimiento del laboratorio

Requisito de la guía	Cumplimiento en el proyecto
Programa enfocado en reconocimiento y clasificacion de objetos	Cumplido con YOLOv8.
Descripcion general del programa	Incluida en informe y UI.
Explicacion del procesamiento de imagenes	Incluida en seccion de funcionamiento.
Funciones utilizadas y creadas	Incluidas en tabla de funciones.
Tecnologias utilizadas	Incluidas en tabla de tecnologias.
Modificaciones realizadas	Incluidas en seccion 7.
Ventajas y desventajas	Incluidas en seccion 8.
Video de funcionamiento	Se incluye guion y pasos para grabacion.
Referencias	Incluidas al final del informe.

10. Cuestionario

10.1. Describan un algoritmo utilizado para el reconocimiento y clasificacion de objetos.

Un algoritmo utilizado es YOLO, que significa You Only Look Once. Este algoritmo pertenece a los detectores de una sola etapa, porque procesa la imagen completa en una sola pasada y predice directamente las cajas

delimitadoras, las clases de los objetos y el nivel de confianza. Esto lo hace adecuado para aplicaciones donde se necesita una respuesta rapida, ya que no requiere separar la imagen en muchas regiones antes de clasificar.

En este proyecto, YOLOv8 recibe la imagen subida por el usuario, la procesa internamente mediante una red neuronal convolucional y devuelve los objetos detectados. Luego el programa filtra los resultados usando un umbral de confianza y muestra las detecciones sobre la imagen mediante cajas y etiquetas.

10.2. Que problemas pueden ocurrir al realizar el reconocimiento y clasificacion de objetos?

Pueden ocurrir varios problemas. Uno de ellos es la presencia de falsos positivos, cuando el sistema detecta un objeto que realmente no existe. Tambien pueden aparecer falsos negativos, cuando un objeto real no es detectado. La baja iluminacion, imagenes borrosas, objetos pequenos, objetos parcialmente tapados, fondos complejos y clases que no fueron consideradas durante el entrenamiento pueden reducir la precision.

Tambien puede haber problemas de rendimiento si se usa un modelo muy grande en una computadora con pocos recursos. Otro punto importante es que un modelo preentrenado no reconoce cualquier objeto del mundo, sino principalmente las clases aprendidas durante su entrenamiento.

11. Conclusiones

El proyecto permite comprobar como la vision computacional puede aplicarse para reconocer y clasificar objetos dentro de imagenes digitales. La UI facilita el uso de la aplicacion, ya que el usuario no necesita escribir comandos para probar una imagen. Ademas, la generacion de tabla, CSV e imagen anotada ayuda a sustentar los resultados del laboratorio.

YOLOv8 resulto adecuado para este caso porque permite detectar objetos de forma rapida y con una implementacion relativamente simple. Sin embargo, la precision depende de la calidad de la imagen, del umbral de confianza y de las clases que el modelo conozca.

12. Referencias

- [1] J. Redmon, S. Divvala, R. Girshick y A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," 2016.
- [2] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," 2014.
- [3] Ultralytics, "YOLOv8 Documentation," documentacion oficial.
- [4] OpenCV, "OpenCV Documentation," documentacion oficial.
- [5] Streamlit, "Streamlit Documentation," documentacion oficial.